

Vite & Gourmand

Documentation Technique

Architecture · Environnement · Modélisation · Diagrammes

Version 1.0 — 2024

Titre Professionnel Développeur Web et Web Mobile

Sommaire

1. Réflexions initiales technologiques 3
2. Configuration de l'environnement de travail 6
3. Modèle Conceptuel de Données (MCD) 9
4. Diagrammes d'utilisation et de séquence 12
5. Documentation du déploiement À compléter

1. Réflexions initiales technologiques

Avant de démarrer le développement, une analyse des besoins techniques a été réalisée afin de choisir les technologies les plus adaptées au projet. Voici les réflexions qui ont guidé ces choix.

1.1 Stack technique imposée par le CDC

Le cahier des charges imposait les technologies suivantes, laissant peu de marge de choix sur les briques principales :

Couche	Technologie imposée	Version utilisée
Front-end	HTML5, CSS (Bootstrap), JavaScript	Bootstrap 5.3.0
Back-end	PHP avec PDO	PHP 8.5.6 + Symfony 7.4.11
BDD relationnelle	MySQL, MariaDB ou PostgreSQL	MariaDB 11.8.6
BDD NoSQL	MongoDB	MongoDB 7.0.34
Déploiement	Fly.io, Heroku, Azure ou Vercel	Fly.io

1.2 Pourquoi Symfony plutôt que PHP natif ?

Le CDC impose PHP avec PDO, mais autorise l'utilisation d'un framework. J'ai choisi Symfony 7 pour les raisons suivantes :

- **Sécurité intégrée** : Symfony fournit nativement la protection CSRF, le hashage des mots de passe (bcrypt), la gestion des sessions et le contrôle d'accès par rôles — des fonctionnalités critiques pour ce projet.
- **Doctrine ORM** : Abstraction de la base de données qui évite l'écriture de requêtes SQL manuelles pour les opérations courantes, tout en permettant du SQL natif pour les vues optimisées.
- **Twig** : Moteur de templates avec échappement automatique des variables (protection XSS native), héritage de templates et filtres puissants.
- **Mailer Component** : Gestion simplifiée des envois d'emails avec support de multiples transports (SMTP, API Resend, Mailtrap...).
- **Structure MVC** : Architecture claire qui facilite la maintenabilité et la séparation des responsabilités.

Comparatif PHP natif vs Symfony :

Critère	PHP natif + PDO	Symfony 7
Sécurité CSRF	À implémenter manuellement	■ Intégré
Hashage mdp	password_hash() manuel	■ UserPasswordHasher
Routing	Gestion manuelle	■ Attributs #[Route]
Templates	PHP pur ou include	■ Twig avec échappement
ORM / BDD	PDO manuel	■ Doctrine ORM
Emails	mail() ou PHPMailer	■ Symfony Mailer
Contrôle accès	Sessions manuelles	■ Security Bundle
Courbe apprentissage	Faible	Moyenne (déjà maîtrisé)

1.3 Choix de MariaDB

Le CDC proposait MySQL, MariaDB ou PostgreSQL. J'ai choisi MariaDB pour les raisons suivantes :

- Compatible à 100% avec MySQL — la migration éventuelle est transparente
- Performances légèrement supérieures à MySQL sur les requêtes complexes
- Fork open-source maintenu activement par la communauté
- Disponible nativement sur le serveur de développement (Fedora Linux)
- Support natif des vues SQL utilisées pour optimiser les requêtes fréquentes

Utilisation des vues SQL :

Pour optimiser les performances et simplifier les requêtes fréquentes, 5 vues SQL ont été créées :

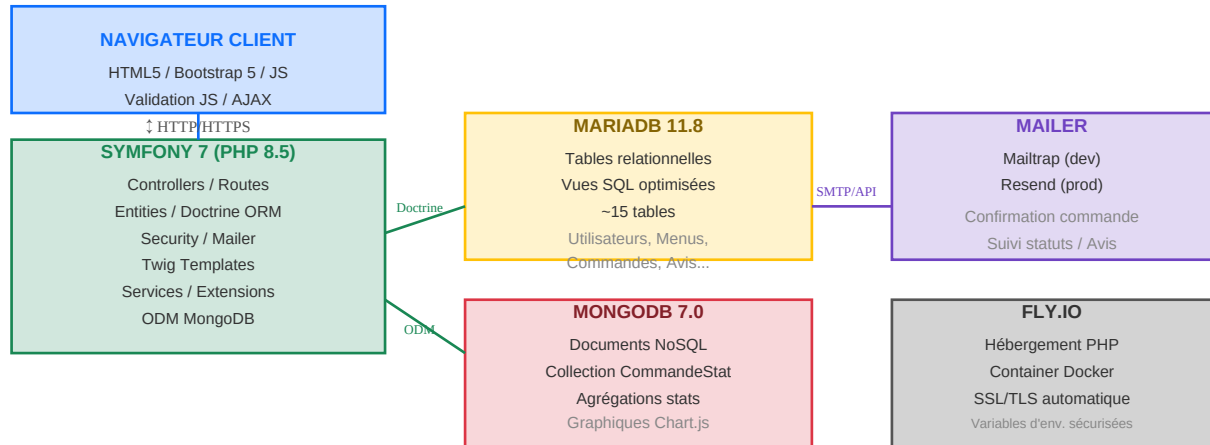
Vue	Rôle
v_menus_actifs	Menus actifs avec nombre de plats — utilisée sur la page liste et accueil
v_commandes_actives	Commandes non terminées/annulées — dashboard employé
v_avis_publies	Avis validés avec pseudo auteur — page accueil et liste avis
v_note_moyenne_menu	Note moyenne par menu — vue détaillée menu
v_dashboard_employe	KPI agrégés (compteurs par statut) — dashboard

1.4 Pourquoi MongoDB pour les statistiques ?

Le CDC impose l'utilisation d'une base NoSQL MongoDB pour les statistiques. Voici pourquoi ce choix est pertinent pour ce cas d'usage :

- **Flexibilité du schéma** : Les documents de statistiques peuvent évoluer sans migration de schéma.
- **Agrégation puissante** : Le pipeline d'agrégation MongoDB permet des calculs de statistiques complexes (SUM, GROUP BY, SORT) directement en base.
- **Séparation des préoccupations** : Les données transactionnelles (commandes) restent dans MariaDB tandis que les données analytiques (stats) sont dans MongoDB — architecture CQRS simplifiée.
- **Document CommandeStat** : Chaque commande passée génère un document MongoDB avec menuId, menuTitre, date, nombreCommandes, prixTotal — parfait pour les graphiques.

1.5 Architecture générale de l'application



Architecture MVC Symfony — Le navigateur communique via HTTP avec Symfony qui orchestre les accès à MariaDB (Doctrine), MongoDB (ODM), et le service Mailer. L'application est déployée sur Fly.io.

2. Configuration de l'environnement de travail

2.1 Système et matériel

Élément	Détail
Système d'exploitation	Fedora Linux
Serveur web	Symfony CLI (serveur de développement intégré)
PHP	PHP 8.5.6
Composer	Gestionnaire de dépendances PHP
MariaDB	11.8.6 — base de données relationnelle
MongoDB	7.0.34 — base de données NoSQL
Git	Versioning du code source
Éditeur de code	VS Code / PhpStorm
Navigateur de test	Firefox Developer Edition + Chrome

2.2 Installation du projet

Étapes pour installer et lancer le projet en local :

Étape 1 — Cloner le dépôt

```
git clone https://github.com/[USERNAME]/vite_gourmand.git cd vite_gourmand
```

Étape 2 — Installer les dépendances

```
composer install
```

Étape 3 — Configurer l'environnement

```
cp .env .env.local # Éditer .env.local avec les paramètres locaux
```

Étape 4 — Créer la base de données

```
php bin/console doctrine:database:create
```

Étape 5 — Importer la structure SQL

```
mysql -u user -p vite_gourmand < sql/structure.sql
```

Étape 6 — Charger les données de test

```
php bin/console doctrine:fixtures:load --append
```

Étape 7 — Lancer le serveur

```
symfony serve
```

2.3 Variables d'environnement (.env.local)

Le fichier `.env.local` (non versionné) contient les paramètres sensibles de l'application :

```
# Environnement APP_ENV=dev APP_SECRET=votre_secret_aleatoire_32_caracteres # Base de données
MariaDB DATABASE_URL="mysql://user:password@127.0.0.1:3306/vite_gourmand
?serverVersion=11.8.6-MariaDB&charset=utf8mb4" # MongoDB
MONGODB_URL="mongodb://127.0.0.1:27017" MONGODB_DB="vite_gourmand" # Mailer (Mailtrap pour le
développement) MAILER_DSN=smtp://identifiant:motdepasse@sandbox.smtp.mailtrap.io:2525
```

2.4 Extensions PHP requises

Extension	Usage	Obligatoire
pdo	Abstraction base de données	■ Oui
pdo_mysql	Connexion MariaDB/MySQL	■ Oui
mongodb	Connexion MongoDB	■ Oui
intl	Internationalisation Symfony	■ Oui
mbstring	Manipulation chaînes multi-octets	■ Oui
xml	Traitement XML	■ Oui
curl	Requêtes HTTP (Mailer API)	■ Oui
openssl	Chiffrement / SSL	■ Oui

2.5 Dépendances Composer principales

Package	Version	Rôle
symfony/framework-bundle	^7.4	Noyau Symfony
symfony/security-bundle	^7.4	Authentification et autorisation
symfony/mailer	^7.4	Envoi d'emails
symfony/twig-bundle	^7.4	Moteur de templates
doctrine/orm	^3.0	ORM pour MariaDB
doctrine/mongodb-odm-bundle	^5.0	ODM pour MongoDB
doctrine/doctrine-fixtures-bundle	^3.0	Données de test (dev)
symfony/password-hasher	^7.4	Hashage bcrypt des mots de passe

3. Modèle Conceptuel de Données (MCD)

Le MCD présente les entités principales de la base de données relationnelle MariaDB et leurs relations. La base comprend 15 tables organisées autour de 4 domaines fonctionnels.

3.1 Entités et attributs principaux

UTILISATEUR ■ id (PK) - nom - prenom - mail (UK) - gsm - adresse - mdp_hash - role - actif - date_creation	MENU ■ id_menu (PK) - titre - description - theme - regime - nb_personnes_min - prix - stock - conditions - actif - date_creation
PLAT ■ id_plat (PK) - nom - type - description - actif	COMMANDE ■ id_commande (PK) ■ id_utilisateur (FK) ■ id_menu (FK) - nb_personnes - adresse - date_prestation - prix_menu - reduction - frais_livraison - prix_total - statut - date_creation
AVIS ■ id_avis (PK) ■ id_commande (FK) ■ id_utilisateur (FK) - note - commentaire - statut - date_creation	ALLERGENE ■ id_allergene (PK) - nom (UK)
IMAGE_MENU ■ id_image (PK) ■ id_menu (FK) - url - alt - ordre	HISTORIQUE_STATUT ■ id_historique (PK) ■ id_commande (FK) ■ id_utilisateur (FK) - statut - changed_at - commentaire

3.2 Relations entre entités

Relation	Type	Description
UTILISATEUR → COMMANDE	1,N	Un utilisateur peut passer plusieurs commandes
MENU → COMMANDE	1,N	Un menu peut être commandé plusieurs fois
MENU ↔ PLAT	N,N	Un menu contient plusieurs plats, un plat peut être dans plusieurs menus (table m
PLAT ↔ ALLERGENE	N,N	Un plat peut avoir plusieurs allergènes, un allergène peut être dans plusieurs plats
MENU → IMAGE_MENU	1,N	Un menu peut avoir plusieurs images (galerie)
COMMANDE → AVIS	1,1	Une commande terminée peut avoir au maximum un avis
COMMANDE → HISTORIQUE_STATUT	1,N	Chaque changement de statut est tracé dans l'historique
ENTREPRISE → HORAIRE	1,7	Une entreprise a 7 horaires (un par jour de la semaine)

3.3 Document MongoDB — CommandeStat

En parallèle de la base relationnelle, MongoDB stocke les statistiques de commandes dans une collection CommandeStat :

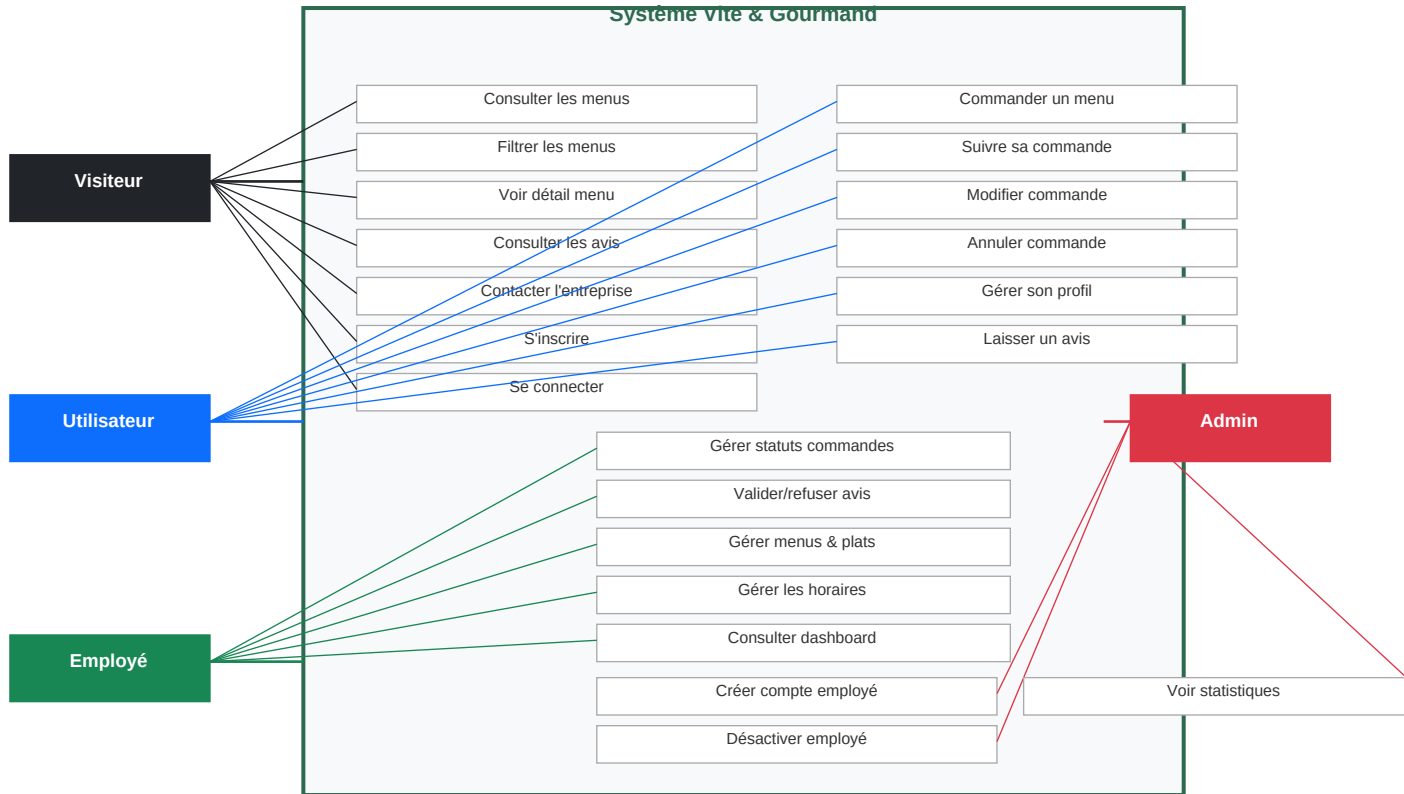
```
{ "_id": ObjectId("..."), "menuId": 1, "menuTitre": "Menu Noël Prestige", "date":  
  ISODate("2024-12-15T14:30:00Z"), "nombreCommandes": 1, "prixTotal": "320.00", "dateCreation":  
  ISODate("2024-12-15T14:30:00Z") }
```

Ce document est créé automatiquement à chaque nouvelle commande validée via le service CommandeStatService. Les agrégations MongoDB permettent ensuite de calculer le nombre de commandes et le chiffre d'affaires par menu pour les graphiques.

4. Diagrammes d'utilisation et de séquence

4.1 Diagramme des cas d'utilisation

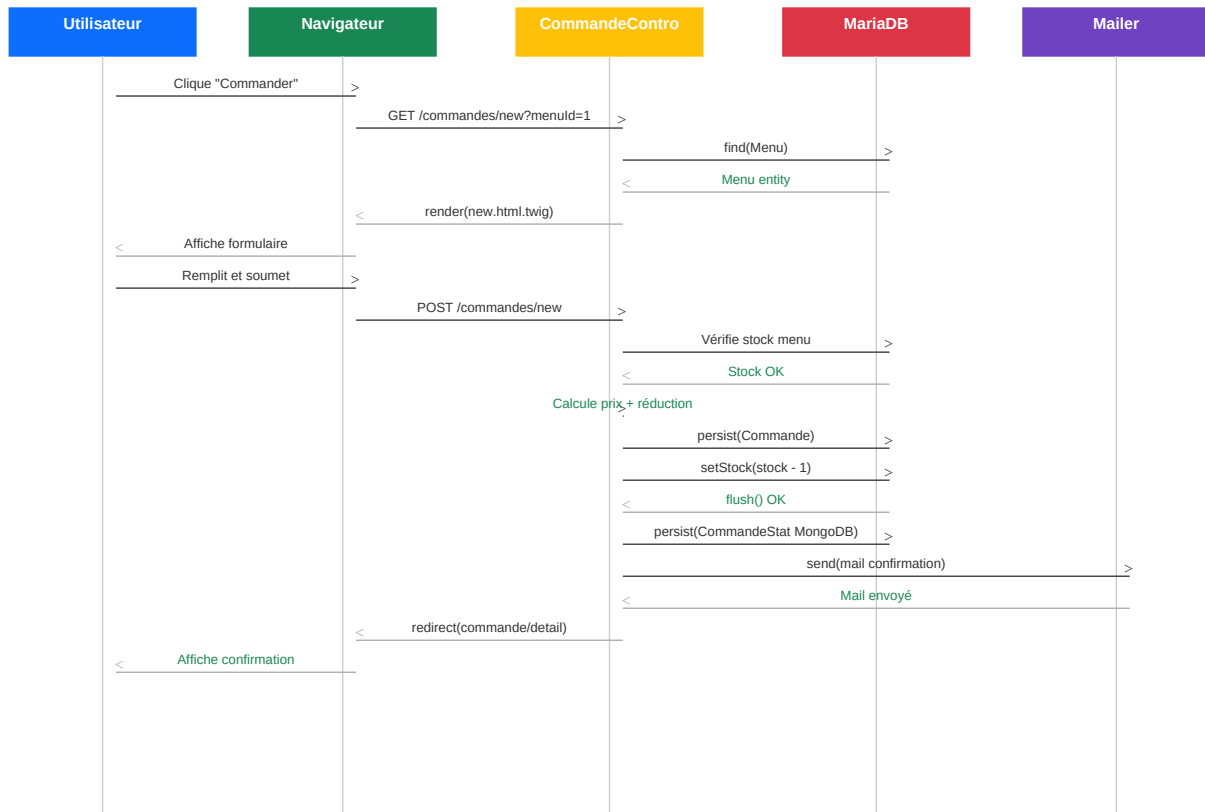
Ce diagramme présente les interactions entre les acteurs et les fonctionnalités de l'application. Quatre acteurs sont identifiés : Visiteur, Utilisateur, Employé et Administrateur.



Les acteurs héritent des droits : Utilisateur $\supset$ Visiteur, Employé $\supset$ Utilisateur, Administrateur $\supset$ Employé.

4.2 Diagramme de séquence — Passage d'une commande

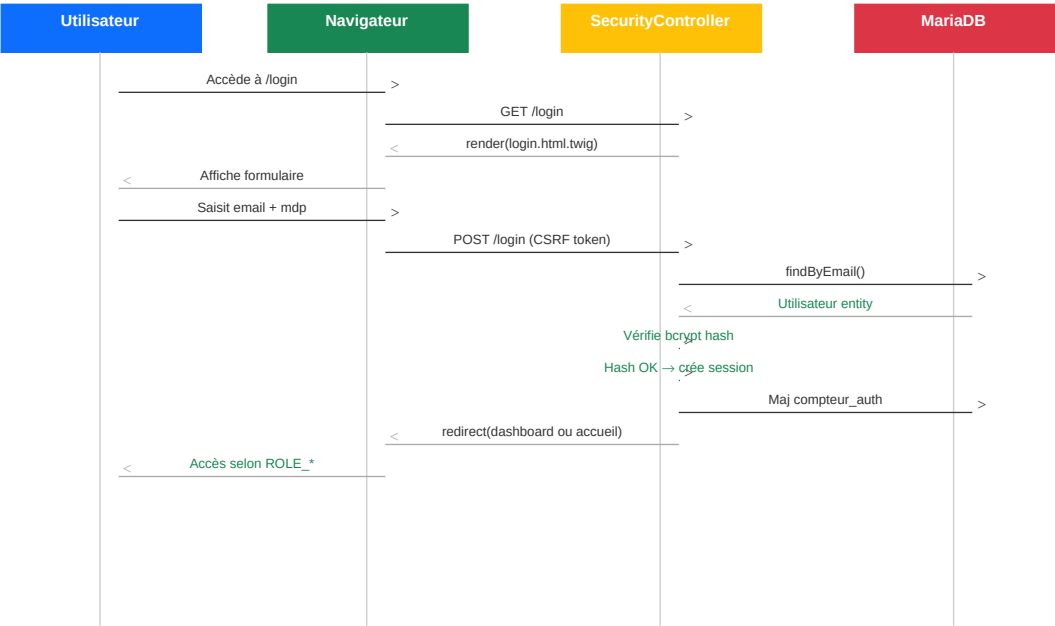
Ce diagramme illustre les interactions entre les composants lors du passage d'une commande par un utilisateur authentifié.



Flux de passage de commande : vérification stock → calcul prix → persistance BDD → enregistrement MongoDB → envoi mail → redirection.

4.3 Diagramme de séquence — Authentification

Ce diagramme illustre le flux d'authentification et de contrôle d'accès.



Sécurité : token CSRF vérifié à chaque soumission, mot de passe comparé via bcrypt, redirection selon le rôle (ROLE_ADMIN → dashboard rouge, ROLE_EMPLOYE → dashboard vert, ROLE_USER → accueil).